Sección A – Arquitectura y Hoja de Ruta

---

1. Arquitectura Objetivo End-to-End — Canal Digital Directo Multi-País

Visión General

La arquitectura objetivo está diseñada para un Canal Digital Directo de seguros multi-país, donde Salesforce actúa como el sistema de registro central y MuleSoft Anypoint Platform como el middleware de integración universal. Toda petición entrante desde canales digitales y toda petición saliente hacia sistemas externos fluye a través de la capa MuleSoft.

La arquitectura responde a cuatro pilares no funcionales:

- **Alta Disponibilidad:** Sin punto único de fallo; despliegue activo-activo en múltiples zonas de disponibilidad
- **Escalabilidad:** Escalado horizontal en la capa de runtime de MuleSoft; límites de governor de Salesforce gestionados mediante aislamiento por bulkhead
- **Resiliencia:** Reintento con backoff exponencial, circuit breaker y bulkhead aplicados en la capa de integración
- **Observabilidad:** Logging unificado, trazas distribuidas (OpenTelemetry) y dashboards en tiempo real sobre todos los flujos de integración

---

Capas de la Arquitectura

Capa 1 — Canales Digitales (Consumidores)

- Portal web (autoservicio del cliente)
- Aplicación móvil
- Portal de agentes y corredores
- Socios externos (APIs)

Todos los canales se comunican exclusivamente a través de las Experience APIs de MuleSoft por HTTPS/REST. Ningún canal tiene acceso directo a Salesforce ni a los sistemas de backend.

Capa 2 — Integración API-Led con MuleSoft (Middleware)

Modelo de conectividad API-led de tres capas:

Experience APIs

- Adaptadores específicos por canal: uno por tipo de canal (web, móvil, corredor, socio)
- Maneja autenticación (OAuth 2.0 / Connected App), validación de peticiones y formateo de respuestas
- Aplica rate limiting y throttling por canal
- Inyecta correlation IDs y contexto de traza (OpenTelemetry) en cada petición

Process APIs

- Capa de orquestación: compone múltiples llamadas a System APIs para construir operaciones de negocio
- Implementa lógica de negocio: orquestación de cotizaciones, emisión de pólizas, ingreso de siniestros
- Aplica patrones de resiliencia: reintento con backoff exponencial + jitter, circuit breaker, validación de clave de idempotencia

- Enruta operaciones de larga duración al canal de mensajería asíncrona (según ADR-002)
- Gestiona manejo de errores y respuestas de fallback

System APIs

- Conectores delgados hacia sistemas de backend — sin lógica de negocio
- Salesforce System API: expone objetos de Salesforce (Account, Opportunity, Policy__c, Claim__c) como recursos REST
- Payment Gateway System API
- Document Generation System API
- Conectores a sistemas legados por país

Capa 3 — Sistemas Core

- **Salesforce:** Core de seguros (pólizas, siniestros, clientes, productos) — sistema de registro
- **Pasarela de Pagos:** Recaudo de primas y reembolsos
- **Servicio de Documentos:** Pólizas, certificados, informes de siniestros
- **Servicio de Notificaciones:** Email, SMS, notificaciones push
- **Sistemas por País:** Sistemas regulatorios locales, proveedores de pago locales

Capa 4 — Capa de Mensajería Asíncrona

Para flujos de larga duración y alto volumen (según ADR-002):

- Broker de mensajes (Anypoint MQ o equivalente cloud-native)
- Cola de mensajes fallidos (Dead Letter Queue - DLQ) para eventos con fallo de procesamiento
- Capacidad de replay de eventos para recuperación ante incidentes
- Consumer groups por país para aislar fallos de procesamiento

Capa 5 — Plataforma de Observabilidad

- **Trazas Distribuidas:** OpenTelemetry SDK en runtimes de MuleSoft; contexto de traza propagado via headers W3C Trace Context en límites síncronos y asíncronos
- **Logging Centralizado:** Logs JSON estructurados agregados en plataforma de gestión de logs (Splunk, ELK o Anypoint Monitoring)
- **Métricas:** Tasa de error, latencia promedio y en el percentil 95 (tiempo que tarda el 95% de las peticiones), estado del circuit breaker, tasa de reintentos, profundidad de cola, retraso del consumidor de mensajes — por API y por país
- **Alertas:** Alertas basadas en el margen de error permitido por el acuerdo de nivel de servicio (SLO) que activan respuesta del equipo de operaciones

---

Patrones de Integración Aplicados

Reintento con Backoff Exponencial y Jitter

Aplicado en la capa Process API para fallos transitorios (timeouts de red, respuestas 5xx de System APIs). Evita el thundering herd durante la recuperación del backend. Fórmula: `espera = min(cap, base * 2^intento) + jitter_aleatorio`.

Circuit Breaker

Cada conector de System API está envuelto por un circuit breaker. Tras un umbral de fallos configurable, el circuito se abre y retorna una respuesta de fallo rápido — protegiendo a Salesforce y otros backends de sobrecarga en cascada. Estados: Cerrado → Abierto → Semi-abierto → Cerrado.

Idempotencia

Toda operación mutante (emisión de póliza, procesamiento de pago) requiere una clave de idempotencia en el header de la petición. La capa Process API deduplica peticiones dentro de una ventana de tiempo configurable, evitando registros duplicados en Salesforce por peticiones reintentadas.

Bulkhead (Mamparo)

El patrón bulkhead aplica aislamiento de recursos en dos niveles:

1. **Capa Experience API:** Cada canal tiene su propio pool de hilos y límite de rate limiting. Si el portal web recibe un pico de tráfico o un ataque, solo su Experience API se ve afectada — la app móvil y el portal de corredores siguen operando normalmente. Ningún canal puede agotar los recursos de otro.
2. **Capa Process API / Sistema:** Los thread pools de MuleSoft están particionados por país y por nivel de criticidad del flujo. Un aluvión de peticiones para Colombia no agota los hilos que atienden a Venezuela. Los flujos críticos (cotización, pago) están aislados de los flujos batch y de reportes.

El resultado es que un fallo o pico en cualquier canal o país queda contenido — no se propaga al resto del sistema.

Caché

- **Caché de cotizaciones:** Catálogo de productos y precios cacheado en la capa Experience API (TTL: 5 minutos) — reduce llamadas a la API de Salesforce para peticiones de cotización de alta frecuencia
- **Caché de tokens:** Tokens de acceso OAuth cacheados hasta su expiración — elimina el overhead de autenticación por petición
- **Caché de datos de referencia:** Configuración por país, catálogo de productos, reglas de cobertura — refrescados por schedule, no por petición

Mensajería Asíncrona

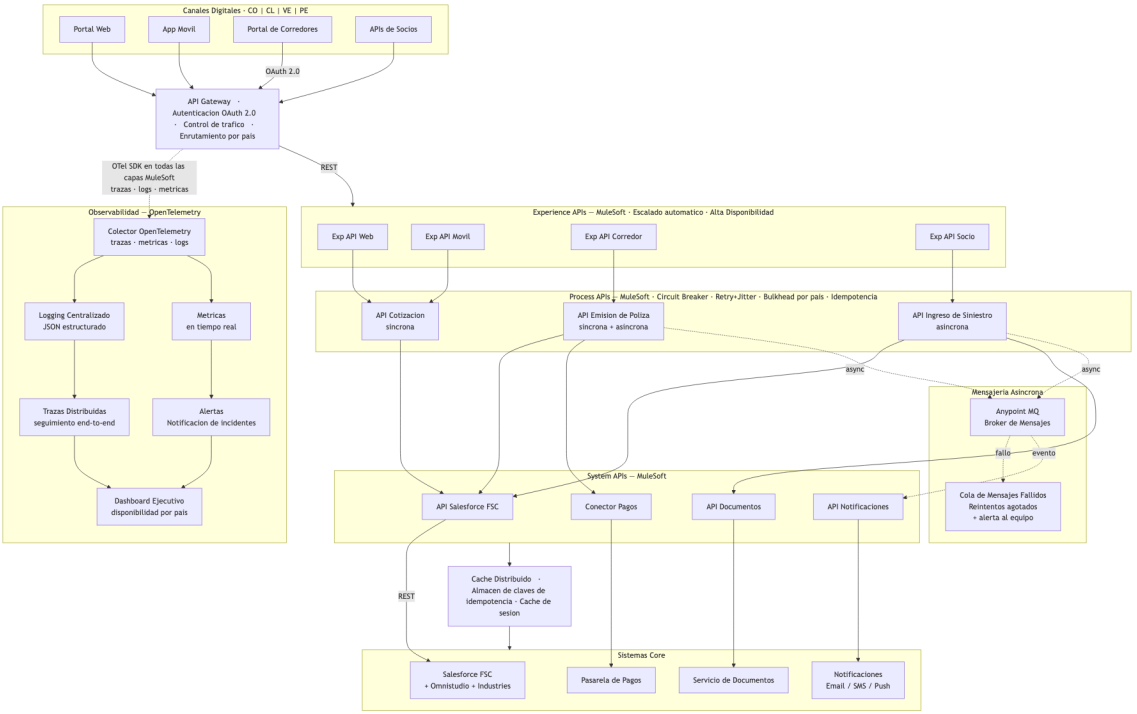
La emisión de pólizas, notificación de siniestros y sincronización de datos entre países se enrutan a la capa de mensajería asíncrona (según ADR-002). Los productores (Process APIs) publican eventos y retornan un acuse de recibo inmediatamente. Los consumidores procesan eventos de forma independiente con soporte de reintento y DLQ.

Alta Disponibilidad y Escalabilidad

- MuleSoft CloudHub 2.0 (o Runtime Fabric on-premises): despliegue activo-activo en múltiples zonas de disponibilidad
- Reglas de auto-escalado basadas en utilización de CPU y profundidad de cola de mensajes
- Volumen de llamadas a la API de Salesforce gestionado mediante aislamiento por bulkhead — cada país tiene una aplicación conectada de Salesforce (Connected App) dedicada con su propia asignación de límites de API
- Load balancer con health checks en la capa Experience API — workers no saludables removidos de la rotación sin downtime
- HA en la capa de base de datos para el almacén de claves de idempotencia (caché distribuido, p. ej. Redis) con replicación

---

2. Diagrama de Arquitectura



---

3. Hoja de Ruta Técnica — 12 Semanas

Tres flujos de trabajo paralelos ejecutados durante 12 semanas:

Track 1 — Confiabilidad

Semana	Actividad
1	Auditar configuraciones actuales de circuit breaker y reintento en todas las APIs de MuleSoft
2	Definir SLOs por API (disponibilidad, latencia p95, tasa de error)
3	Implementar aislamiento de thread pool por bulkhead por país en MuleSoft
4	Desplegar validación de clave de idempotencia para todas las Process APIs mutantes
5	Implementar reintento con backoff exponencial + jitter en todas las llamadas a System APIs
6	Revisión de confiabilidad: pruebas de fallo controlado (chaos engineering) en escenarios de circuit breaker
7	Implementar capa de caché: catálogo de productos, tokens OAuth
8	Pruebas de carga: validar auto-escalado y aislamiento por bulkhead bajo tráfico pico
9	Implementar procesamiento y alertas de Dead Letter Queue
10	Pruebas de fallo controlado: simular degradación de Salesforce; validar circuit breaker y fallback

11	Validación de HA multi-país: simular fallo de zona de disponibilidad
12	Cierre: todos los SLOs cumplidos, todos los caminos críticos probados

Track 2 — Modernización de Integración

Semana	Actividad
1	Inventariar todas las System APIs existentes e identificar duplicación entre países
2	Establecer estándares de diseño de API y política de versionamiento semántico (ej. v1.2.3)
3	Refactorizar conectores duplicados por país en System APIs compartidas
4	Migrar las 3 integraciones de mayor tráfico al nuevo patrón de System API
5	Implementar mensajería asíncrona para el flujo de emisión de pólizas
6	Implementar mensajería asíncrona para el flujo de notificación de siniestros
7	Publicar plantillas reutilizables de Experience API en Anypoint Exchange
8	Migrar segundo lote de integraciones por país al patrón compartido de System API
9	Implementar idempotencia en el consumidor de mensajes (deduplicación asíncrona)
10	Revisión de gobierno de APIs: deprecar integraciones directas legadas
11	Completar librería de assets en Anypoint Exchange: todas las APIs reutilizables documentadas
12	Cierre: sin integraciones directas a Salesforce que eviten MuleSoft

Track 3 — Observabilidad y Operaciones

Semana	Actividad
1	Desplegar OpenTelemetry collector; instrumentar las 5 Process APIs críticas principales
2	Implementar logging JSON estructurado con propagación de correlation ID
3	Construir primer dashboard de métricas: tasa de error, latencia, estado del circuit breaker
4	Definir reglas de alerta basadas en el margen de error permitido por el acuerdo de nivel de servicio (SLO)
5	Instrumentar flujos asíncronos: profundidad de cola, retraso del consumidor de mensajes, tasa de DLQ
6	Primer runbook on-call: circuit breaker abierto, pico de DLQ, degradación de latencia
7	Trazas distribuidas end-to-end: propagación de traza en límites síncronos y asíncronos
8	Revisión de SLOs: ajustar umbrales con base en 4 semanas de datos reales de operación
9	Informe de planeación de capacidad: límites de API de Salesforce por país, utilización de workers

10	Finalizar runbook on-call para todos los escenarios P1/P2
11	Informe ejecutivo de observabilidad: cumplimiento de SLOs, tendencias de incidentes
12	Cierre: trazas, logs, métricas, alertas y runbooks completamente operativos

Resumen de Flujos de Trabajo

Confiabilidad — Semanas 1–12: Establecer y hacer cumplir patrones de resiliencia (circuit breaker, reintento, bulkhead, idempotencia, caché) en todos los flujos de integración. Validar bajo condiciones reales de fallo.

Modernización de Integración — Semanas 1–12: Consolidar integraciones fragmentadas por país en una capa de System APIs compartida. Eliminar el acceso directo a Salesforce. Publicar assets reutilizables en Anypoint Exchange. Adoptar mensajería asíncrona para flujos de larga duración.

Observabilidad y Operaciones — Semanas 1–12: Instrumentar todos los flujos críticos con OpenTelemetry. Construir dashboards, definir SLOs, configurar alertas y producir runbooks operativos. Transitar de operaciones reactivas a proactivas.

Sección B – Framework de Integración Reutilizable

Visión General

El framework de integración es un módulo Python reutilizable (`src/framework.py`) que provee un punto de entrada único para todas las llamadas HTTP salientes de un servicio. Toda integración pasa por este cliente — los patrones de resiliencia se aplican una sola vez aquí, sin duplicarlos en cada punto de integración.

Componentes

1. Configuración Centralizada (`IntegrationConfig`)

Todos los parámetros ajustables se declaran en un único dataclass. En producción, estos valores se cargan desde variables de entorno o un servicio de configuración (propiedades de MuleSoft, AWS Parameter Store, etc.).

```
config = IntegrationConfig(
    max_retries=3,
    base_backoff_seconds=0.5,
    max_backoff_seconds=10.0,
    jitter_factor=0.3,
    timeout_seconds=5.0,
    circuit_breaker_failure_threshold=5,
    circuit_breaker_recovery_seconds=30.0,
    service_name="api-emision-polizas"
)
```

Decisión de diseño: Centralizar la configuración evita números mágicos dispersos en el código y permite ajuste en runtime sin cambios de código.

---

2. Reintento con Backoff Exponencial y Jitter

Fórmula: `espera = min(cap, base × 2^intento) + jitter_aleatorio`

Intento	Demora base	Con cap	+ Jitter (30%)	Espera aprox.
0 (1er reintento)	0.5s	0.5s	±0.15s	~0.5–0.65s
1 (2do reintento)	1.0s	1.0s	±0.30s	~1.0–1.30s
2 (3er reintento)	2.0s	2.0s	±0.60s	~2.0–2.60s

Por qué backoff exponencial: Las demoras progresivas le dan al sistema downstream tiempo creciente para recuperarse entre intentos. Un intervalo de reintento fijo puede sobrecargar un backend en recuperación parcial.

Por qué jitter: Cuando múltiples clientes fallan simultáneamente (p. ej., una interrupción breve de red), todos reintentan en el mismo intervalo si no hay jitter — creando una avalancha de reintentos simultáneos que impide al backend en recuperación estabilizarse. El jitter distribuye los reintentos en el tiempo, suavizando la carga.

Por qué 3 reintentos: Basado en las ventanas de fallo transitorio observadas — la mayoría de errores transitorios se resuelven en 2 reintentos. Un 4to reintento agrega latencia sin beneficio proporcional y puede comprometer el SLO del servicio llamador.

---

3. Circuit Breaker

Máquina de tres estados que protege los sistemas downstream de sobrecarga durante ventanas de fallo:

```
CERRADO ──(umbral de fallos)──> ABIERTO ──(ventana de recuperación)──> SEMI-ABIERTO
   ^                                                                        |
   └──────────────────────(llamada de prueba exitosa)──────────────────────┘
```

Estado	Comportamiento
Cerrado	Operación normal. Los fallos se cuentan.
Abierto	Todas las llamadas se rechazan inmediatamente (fallo rápido). El sistema downstream descansa.
Semi-abierto	Se permite una llamada de prueba. Éxito → Cerrado. Fallo → Abierto.

Por qué el circuit breaker es importante: Sin él, las tormentas de reintentos de múltiples clientes golpean un sistema downstream en fallo — impidiéndole recuperarse. El circuit breaker rompe este ciclo deteniendo las llamadas completamente durante la ventana de recuperación.

Configuración: 5 fallos → Abierto; 30s de ventana de recuperación antes de la llamada de prueba Semi-abierto.

---

4. Timeout por Petición

Cada llamada HTTP está acotada por un timeout configurable (por defecto: 5 segundos). Esto evita que un sistema downstream lento bloquee indefinidamente un hilo — crítico en flujos síncronos orientados al usuario, donde el presupuesto total de tiempo de respuesta está acotado por el SLA del servicio llamador.

Decisión de diseño: El timeout siempre se configura menor que el timeout del servicio llamador, para garantizar que el framework pueda manejar el fallo con gracia antes de que venza el plazo del llamador.

---

5. Soporte de Clave de Idempotencia

Las operaciones mutantes (POST, PUT, PATCH) aceptan una clave de idempotencia. El framework almacena las respuestas exitosas indexadas por ese valor. Las peticiones duplicadas retornan la respuesta cacheada inmediatamente sin hacer una llamada de red.

```
respuesta = cliente.call(
    url="https://salesforce-api/polizas",
    method="POST",
    body={"account_id": "001...", "producto": "vida"},
    idempotency_key="emision-poliza-req-abc123"
)
```

Por qué la idempotencia es esencial para el reintento: Sin ella, una petición que expiró por timeout pero que fue procesada por el sistema downstream (p. ej., Salesforce creó la póliza) generaría un registro duplicado al reintentar. La clave de idempotencia hace que los reintentos sean seguros.

Implementación en producción: El almacén sería un caché distribuido (Redis) compartido entre todos los workers de MuleSoft, con un TTL que coincide con la ventana máxima esperada de reintentos.

---

6. Logging Estructurado Unificado

Cada entrada de log es un objeto JSON con un esquema consistente:

```
{
  "nivel": "advertencia",
  "evento": "llamada_fallida_reintentando",
  "timestamp": "2026-03-28T19:00:00Z",
  "url": "https://salesforce-api/polizas",
  "metodo": "POST",
  "intento": 1,
  "error": "HTTP 503 de ...: Servicio no disponible",
  "espera_segundos": 1.23,
  "trace_id": "4bf92f3577b34da6a3ce929d0e0e4736"
}
```

Por qué logging estructurado: Los logs en texto libre no pueden ser analizados o consultados de forma confiable en plataformas de agregación de logs (Splunk, ELK). Un esquema JSON fijo permite filtrar por `trace_id`, `clave_idempotencia` o tipo de `evento` en millones de entradas — esencial para el diagnóstico de incidentes.

---

7. Propagación de Trazas con OpenTelemetry

Cada petición saliente lleva encabezados de traza (`traceparent` , `tracestate`) siguiendo el estándar W3C Trace Context — el formato universal para propagar el ID de traza entre servicios. Esto permite a la plataforma de observabilidad unir toda la cadena de llamadas — desde el canal digital a través de las Process APIs de MuleSoft hasta Salesforce y de regreso — incluso a través de límites de mensajería asíncrona.

```
traceparent: 00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01
tracestate:  sura=00f067aa0ba902b7
```

En producción: Reemplazar la clase `TraceContext` liviana con el paquete `opentelemetry-sdk` . El framework crea un registro por cada llamada saliente (span), captura estado y latencia, y los envía al OpenTelemetry Collector (que alimenta herramientas como Jaeger o Datadog para visualización).

Ejemplo de Uso

```python
from framework import IntegrationClient, IntegrationConfig, TraceContext

config = IntegrationConfig(
    max_retries=3,
    timeout_seconds=5.0,
    circuit_breaker_failure_threshold=5,
    service_name="api-emision-polizas"
)

cliente = IntegrationClient(config=config, circuit_name="salesforce-system-api")
traza = TraceContext()  # Span raíz para esta petición

respuesta = cliente.call(
    url="https://salesforce-system-api/polizas",
    method="POST",
    body={"account_id": "001ABC", "codigo_producto": "VIDA_COL"},
    idempotency_key="emision-2026-03-28-001ABC-VIDA",
    trace_context=traza
)
```

Resumen de Decisiones de Diseño

Decisión	Justificación
Único punto de entrada `IntegrationClient`	Patrones de resiliencia aplicados una vez, no duplicados por integración
Backoff exponencial con jitter	Evita thundering herd; da al sistema downstream tiempo progresivo de recuperación
Circuit breaker separado del reintento	El reintento maneja errores transitorios; el circuit breaker maneja interrupciones sostenidas — modos de fallo distintos requieren respuestas distintas

Idempotencia a nivel de framework	Reintentos seguros para todas las operaciones mutantes sin que el llamador lo gestione
Logging JSON estructurado	Logs legibles por máquina para consultas confiables en plataformas de agregación
Encabezados de traza estándar (W3C)	Formato universal de propagación de ID de traza; compatible con todas las plataformas de observabilidad principales
Almacén de idempotencia en memoria (demo)	Equivalente en producción: Redis con TTL; misma interfaz, respaldo distribuido

Sección C – Servicio Demo y Prueba de Confiabilidad

Visión General

El servicio demo simula una **Process API de Emisión de Pólizas** que llama a una **Salesforce System API** usando el framework de integración de la Sección B. El sistema upstream es intencionalmente inestable — simula las condiciones de fallo que una integración de seguros real enfrenta en producción.

El demo no requiere dependencias externas y se ejecuta con un solo comando.

Cómo Ejecutarlo

Requisitos: Python 3.8+

```
# Desde la raíz del repositorio
python src/demo.py
```

Sin paquetes adicionales. El servidor upstream inestable corre en el mismo proceso en el puerto 9999.

Qué Demuestra el Demo

Arquitectura

```
Servicio Demo (API de Emisión de Pólizas)
    │
    ├── IntegrationClient (framework.py)
    │       ├── Verificación de idempotencia
    │       ├── Verificación del circuit breaker
    │       ├── Llamada HTTP con timeout (5s)
    │       ├── Reintento + backoff exponencial + jitter
    │       └── Logging estructurado + propagación de traza
    │
    └──> Upstream Inestable (Salesforce System API simulada :9999)
             ├── 30% éxito (HTTP 200)
             ├── 60% fallo transitorio (HTTP 503)
             └── 10% timeout (respuesta después de 8s > timeout cliente 5s)
```

Escenarios de Fallo Demostrados

Escenario 1 — Reintento bajo Fallos Transitorios

El upstream retorna HTTP 503 de forma aleatoria. El framework reintenta hasta 3 veces con backoff exponencial + jitter antes de rendirse.

Ejemplo de salida en consola:

```
{"nivel": "advertencia", "evento": "llamada_fallida_reintentando",
 "url": "http://localhost:9999/v1/polizas",
 "intento": 0, "error": "HTTP 503 de ...", "espera_segundos": 0.52, "trace_id":
"4bf92f..."}

{"nivel": "advertencia", "evento": "llamada_fallida_reintentando",
 "url": "http://localhost:9999/v1/polizas",
 "intento": 1, "error": "HTTP 503 de ...", "espera_segundos": 1.18, "trace_id":
"4bf92f..."}

{"nivel": "info", "evento": "llamada_exitosa",
 "url": "http://localhost:9999/v1/polizas", "intento": 2, "trace_id": "4bf92f..."}
```

Cómo la resiliencia mitiga el problema: El cliente reintenta automáticamente. El servicio llamador recibe una respuesta exitosa sin conocer los fallos subyacentes. El backoff progresivo le da tiempo al upstream para recuperarse entre intentos.

Escenario 2 — Timeout por Petición

El upstream duerme por 8 segundos. El cliente aplica un timeout de 5 segundos, falla rápido y reintenta.

Cómo la resiliencia mitiga el problema: Sin timeout, el hilo del cliente se bloquearía por 8+ segundos — violando el SLA del servicio llamador (p. ej., un usuario web esperando una cotización). El timeout fuerza un fallo rápido, permitiendo que el reintento potencialmente alcance un worker sano del upstream.

Escenario 3 — Circuit Breaker se Abre

Tras 3 fallos consecutivos (umbral del demo), el circuito se abre. Las llamadas posteriores se rechazan inmediatamente sin tocar el upstream.

Ejemplo de salida en consola:

```
{"nivel": "advertencia", "evento": "circuit_breaker_abierto",
 "circuit": "salesforce-system-api",
 "cantidad_fallos": 3, "segundos_recuperacion": 10.0}

{"nivel": "advertencia", "evento": "circuit_breaker_rechazado",
 "url": "http://localhost:9999/v1/polizas",
 "circuit": "salesforce-system-api", "trace_id": "7c3a1f..."}
```

Cómo la resiliencia mitiga el problema: El circuit breaker deja de golpear el upstream que ya está fallando, dándole espacio para recuperarse. El llamador recibe un `CircuitBreakerOpenError` rápido y predecible

— en lugar de esperar múltiples timeouts y reintentos.

Escenario 4 — Acierto en Caché de Idempotencia

Una petición con una clave de idempotencia previamente exitosa es reenviada. El framework retorna la respuesta cacheada sin realizar ninguna llamada de red.

Ejemplo de salida en consola:

```
{"nivel": "info", "evento": "cache_idempotencia_hit",
 "clave_idempotencia": "demo-poliza-001", "trace_id": "4bf92f..."}
```

Cómo la resiliencia mitiga el problema: En producción, un cliente móvil podría reintentar una solicitud de emisión de póliza tras una caída de red. Sin idempotencia, Salesforce crearía una póliza duplicada. La clave de idempotencia garantiza que la operación se ejecute exactamente una vez — incluso a través de múltiples reintentos.

Salida de Observabilidad

Cada evento produce una entrada de log JSON estructurado visible en consola. En producción, estas entradas fluyen a la plataforma centralizada de agregación de logs. El campo `trace_id` vincula todas las entradas de log de una misma cadena de petición — a través de reintentos, servicios y límites síncronos/asíncronos.

Equivalentes en Producción

Componente del Demo	Equivalente en Producción
Servidor HTTP inestable en proceso	Salesforce System API bajo degradación
Almacén de idempotencia en memoria	Caché distribuido Redis con TTL
Contexto de traza con `print()`	OpenTelemetry SDK → Collector → Jaeger/Datadog
Proceso Python único	Worker de MuleSoft CloudHub con múltiples réplicas
`CircuitBreakerOpenError`	Error handler de MuleSoft retornando HTTP 503 + header Retry-After

Sección D – Registro de Decisión Técnica

ADR-001: Plataforma de Integración Centralizada vs. Integraciones Descentralizadas por Equipo

Fecha: 2026-03-28 **Estado:** Aceptado **Decisores:** Líder Técnico, Equipo de Arquitectura

---

Contexto

Sura opera un Canal Digital Directo multi-país donde toda petición entrante y saliente fluye a través de una capa de middleware centralizada (MuleSoft Anypoint Platform) que media entre Salesforce (sistema de registro central de seguros) y sistemas externos, socios y canales digitales.

A medida que la organización escala en países y líneas de producto, los equipos enfrentan una presión creciente por entregar integraciones más rápido. Surgen dos modelos en competencia:

- **Opción A:** Mantener y fortalecer la plataforma de integración centralizada de MuleSoft, gestionada por un equipo dedicado de integración.
- **Opción B:** Permitir que los equipos de producto sean dueños y construyan sus propias integraciones directamente, reduciendo la dependencia de un equipo central.

Esta decisión tiene impacto directo en velocidad de entrega, gobierno, resiliencia operativa y mantenibilidad a largo plazo.

Opciones Consideradas

Opción A — Plataforma de Integración Centralizada (API-Led de MuleSoft)

Todas las integraciones se construyen, despliegan y operan a través de MuleSoft Anypoint Platform siguiendo una arquitectura API-led de tres capas:

- **Experience APIs:** Adaptadores específicos por canal (móvil, web, portales de corredores)
- **Process APIs:** Orquestación y lógica de negocio
- **System APIs:** Conectores a Salesforce, sistemas core de seguros y proveedores externos

Un equipo dedicado de integración es dueño de la plataforma, hace cumplir los estándares y publica assets reutilizables en Anypoint Exchange.

Ventajas:

- Punto único de cumplimiento para seguridad, rate limiting y observabilidad
- Assets de API reutilizables reducen la duplicación entre países
- Logging, trazas y manejo de errores consistentes en todas las integraciones
- Cumplimiento y auditabilidad más sencillos (requisitos regulatorios del sector asegurador)
- Patrones de resiliencia (reintento, circuit breaker, idempotencia) aplicados una vez, heredados en todas partes

Desventajas:

- El equipo central se convierte en cuello de botella para los equipos de producto de alta velocidad
- Requiere fuerte gobierno de diseño de API y disciplina de versionamiento
- Mayor inversión inicial en experiencia en la plataforma y herramientas

---

Opción B — Integraciones Descentralizadas por Equipo

Cada equipo de producto construye y opera sus propias integraciones usando la tecnología de su elección. Los equipos son responsables de su propia confiabilidad, seguridad y observabilidad.

Ventajas:

- Los equipos se mueven más rápido sin esperar a un equipo central
- Flexibilidad tecnológica por contexto de equipo
- Menor dependencia organizacional

Desventajas:

- Observabilidad fragmentada — sin vista unificada del estado del sistema
- Brechas de seguridad y cumplimiento entre equipos de diferente madurez
- Duplicación de lógica de integración entre países y canales
- Patrones de resiliencia implementados de forma inconsistente o inexistente
- Alto costo operativo: N equipos operando N stacks de integración
- En una industria regulada (seguros), la propiedad descentralizada incrementa significativamente la complejidad de auditoría

---

Decisión

Recomendamos la Opción A — Plataforma de Integración Centralizada, con ajustes tácticos específicos para reducir cuellos de botella.

Justificación:

1. **El contexto regulatorio exige gobierno.** Las operaciones de seguros en múltiples países requieren pistas de auditoría consistentes, controles de soberanía de datos y cumplimiento de seguridad. Una plataforma centralizada provee un único perímetro de cumplimiento.
2. **La resiliencia a escala requiere consistencia.** Patrones como circuit breakers, idempotencia y reintento con backoff exponencial deben aplicarse de manera uniforme. La propiedad descentralizada produce resiliencia inconsistente — algunos equipos la implementan, otros no.
3. **La reutilización reduce el costo total.** Las System APIs de Salesforce y las Process APIs del core de seguros, construidas una vez, pueden reutilizarse en todos los países y canales. La descentralización las reconstruye para cada equipo.
4. **El cuello de botella es resoluble sin descentralizar.** El dolor real es la velocidad de los equipos, no el modelo centralizado en sí. La solución es un **modelo de contribución abierta interna**: los equipos de producto contribuyen a la plataforma de integración mediante patrones de autoservicio gobernados, plantillas publicadas y conectores reutilizables — mientras el equipo central se enfoca en confiabilidad de la plataforma y gobierno, no en atender tickets de cada equipo.

Consecuencias

Positivas:

- Observabilidad unificada en todos los flujos de integración
- Postura de resiliencia y seguridad consistente
- Reducción de duplicación de conectores de Salesforce y sistemas core
- Reportes de cumplimiento más sencillos entre países

Negativas / Mitigaciones:

- El equipo central debe actuar como equipo de soporte, no de control — proveer plantillas, aceleradores y patrones de autoservicio a los equipos de producto
- El gobierno de la plataforma debe ser liviano — los procesos de revisión pesados anulan el propósito
- Requiere inversión en expertise de MuleSoft y librería de assets en Anypoint Exchange

---

ADR-002: Arquitectura Event-Driven vs. Solicitud-Respuesta Síncrona para Flujos Críticos

Fecha: 2026-03-28 **Estado:** Aceptado **Decisores:** Líder Técnico, Equipo de Arquitectura

---

Contexto

El Canal Digital Directo maneja múltiples tipos de flujos con características distintas:

- **Generación de cotización:** El usuario solicita una cotización de seguro en tiempo real — espera una respuesta en menos de 3 segundos
- **Emisión de póliza:** Dispara procesos downstream en Salesforce, pasarelas de pago y generación de documentos
- **Notificación de siniestro:** Inicia un flujo de trabajo multi-paso entre equipos internos y ajustadores externos
- **Sincronización de datos entre países:** Los datos de pólizas y clientes deben mantenerse consistentes entre instancias de Salesforce por país

La pregunta es: ¿qué flujos deben ser **síncronos (solicitud-respuesta)** y cuáles **event-driven (mensajería asíncrona)**?

Opciones Consideradas

Opción A — Solicitud-Respuesta Síncrona para Todos los Flujos

Todas las operaciones se manejan mediante llamadas HTTP síncronas a través de MuleSoft. El llamador espera una respuesta completa antes de continuar.

Ventajas:

- Modelo de programación simple — más fácil de razonar y depurar
- Retroalimentación de error inmediata al llamador
- No requiere infraestructura adicional de mensajería

Desventajas:

- Acoplamiento fuerte entre sistemas — si el downstream falla, todo el flujo falla
- No puede manejar picos de alto volumen con gracia — la carga se propaga directamente a los backends
- Las operaciones de larga duración (siniestros, generación de documentos) bloquean el hilo del llamador
- Fallos en cascada: una respuesta lenta de Salesforce degrada todo el canal

---

Opción B — Event-Driven para Flujos de Larga Duración y Alto Volumen, Síncrono para Respuestas en Tiempo Real

Un modelo híbrido donde el patrón de comunicación se selecciona según las características del flujo:

Tipo de Flujo	Patrón	Justificación
Generación de cotización	Síncrono	El usuario espera la respuesta; debe ser < 3s
Emisión de póliza	Event-driven (asíncrono)	Flujo de trabajo multi-paso; desacopla el canal del procesamiento backend
Notificación de siniestro	Event-driven (asíncrono)	Inicia un flujo de trabajo de larga duración multi-equipo
Sincronización de datos entre países	Event-driven (asíncrono)	La consistencia eventual es aceptable; el volumen puede ser alto

Autenticación / sesión	Síncrono	Sensible a la seguridad; requiere validación inmediata

Ventajas:

- Desacopla productores de consumidores — el canal permanece responsivo incluso si Salesforce es lento
- Absorbe picos de tráfico mediante buffering en la cola de mensajes
- Habilita patrones de reintento y cola de mensajes fallidos para eventos con error
- Los flujos de trabajo de larga duración no bloquean los hilos orientados al usuario

Desventajas:

- Introduce consistencia eventual — los datos se sincronizan con un pequeño retraso, no en tiempo real; requiere idempotencia y deduplicación
- Modelo operativo más complejo — requiere monitoreo de profundidad de cola y consumer lag
- Más difícil seguir el flujo completo sin IDs de correlación apropiados y trazas distribuidas

---

Decisión

Recomendamos la Opción B — Modelo híbrido: event-driven para flujos de larga duración y alto volumen, síncrono para respuestas orientadas al usuario en tiempo real.

Justificación:

1. **La experiencia del usuario dicta los flujos síncronos.** La generación de cotizaciones y la autenticación no pueden ser asíncronas — los usuarios esperan retroalimentación inmediata. Forzarlos por una cola agrega latencia sin beneficio.
2. **La resiliencia del backend requiere desacoplamiento.** La emisión de pólizas y la notificación de siniestros disparan flujos de trabajo downstream complejos en Salesforce y sistemas externos. Una cadena síncrona significa que cualquier downstream lento o fallido degrada todo el canal. Un enfoque event-driven aísla los fallos al servicio consumidor.
3. **Las colas absorben los picos de tráfico, no los backends.** En un canal digital multi-país, el tráfico pico (lanzamientos de campañas, períodos de renovación) puede saturar los backends síncronos. Las colas de mensajes actúan como amortiguadores, suavizando la carga sin necesidad de escalar los backends más allá de lo necesario.
4. **La idempotencia hace seguro el async.** Cada evento lleva una clave de idempotencia, habilitando replay seguro sin procesamiento duplicado — un requisito ya presente en el framework de integración (Sección B).

Consecuencias

Positivas:

- El canal permanece responsivo ante degradación del backend
- Los flujos de trabajo de larga duración son confiables y reintentables
- Los picos de tráfico no se propagan en cascada hacia fallos del backend
- Cada tipo de flujo usa el patrón de comunicación más adecuado a sus características

Negativas / Mitigaciones:

- La complejidad operativa aumenta — mitigada por observabilidad centralizada (Sección A) con dashboards de profundidad de cola y consumer lag

- La consistencia eventual requiere disciplina de idempotencia — aplicada a nivel del framework de integración
- El trazado end-to-end requiere correlation IDs propagados en límites síncronos y asíncronos — aplicado mediante propagación de trazas de OpenTelemetry (Sección B)